

# Whole project explanation, folder by folder, with data flow and working details

This report is based on the active application inside **my-react-app**. It explains how the frontend, backend, database, and shared state work together, what data is passed between each part, and how the project is organized.

**Frontend: React + Vite**

**Backend: Node + Express**

**Database: MongoDB + Mongoose**

**Auth: JWT + bcrypt**

Scope note: the active app is the nested **my-react-app** project. The root of the workspace contains some older/legacy files, but the running frontend and backend are inside this folder.

## Table of Contents

1. Project overview

2. Folder structure

3. Frontend breakdown

4. Backend breakdown

5. End-to-end data flow

6. API reference

7. Database models

8. Environment variables

9. How to run the project

10. Notes and future improvements

# 1. Project Overview

PharmaHub is a medicine shopping and order management application. It lets a user browse products, search medicines, add items to a cart, register or log in, place an order, and view order history. The frontend is built with React and Vite, while the backend is built with Express and MongoDB.

The project is split into two major parts:

- **Frontend:** handles UI, routing, shared state, and calls to the backend API.
- **Backend:** handles authentication, product lookup, order creation, and database storage.

The app uses **JWT authentication** to protect order-related actions. The frontend stores the logged-in user and token in localStorage so the session survives page refreshes.

# 2. Folder Structure

Below is the practical structure of the active app. This is the part that matters for how the code runs today.

```
my-react-app
├── backend
│   ├── config
│   ├── controllers
│   ├── middleware
│   ├── models
│   ├── routes
│   ├── .env
│   └── server.js
├── frontend
│   ├── index.html
│   ├── package.json
│   └── src
│       ├── assets
│       ├── components
│       ├── context
│       ├── css
│       ├── data
│       ├── pages
│       ├── services
│       ├── App.jsx
│       └── main.jsx
└── package.json
```

## Frontend folders

- **assets**: images and SVG assets used by the UI.
- **components**: reusable pieces like Navbar, ProductCard, ProtectedRoute.
- **context**: global state providers for auth and cart.
- **css**: shared styles, page styles, and component styles.
- **data**: placeholder folder for static data or seed data.
- **pages**: route-based screens like Home, Products, Cart, Orders, Login.
- **services**: API client setup for Axios.

## Backend folders

- **config**: MongoDB connection setup.
- **controllers**: business logic for auth, products, and orders.
- **middleware**: request protection and token verification.
- **models**: Mongoose schemas for users, products, and orders.
- **routes**: Express route definitions and URL grouping.
- **.env**: environment values like database URL, port, and JWT secret.
- **server.js**: main backend entry file.

## 3. Frontend Breakdown

### 3.1 Entry files

- **src/main.jsx** creates the React root and wraps the app with **AuthProvider** and **CartProvider**.
- **src/App.jsx** defines the application routes and places the Navbar above all pages.
- **src/services/api.js** creates the Axios client with base URL `http://localhost:5000/api`.

### 3.2 Shared state

The frontend uses React Context in two places:

- **AuthContext**: stores the logged-in user and exposes `login` and `logout`.
- **CartContext**: stores cart items and exposes actions for `add`, `remove`, `increase quantity`, `decrease quantity`, and `clear cart`.

### 3.3 Components

- **Navbar** shows navigation links, cart count, login/register links, and logout controls.

- **ProductCard** renders a single medicine card and triggers add-to-cart.
- **ProtectedRoute** blocks unauthenticated users from reaching the cart page.

### 3.4 Pages

| Page           | What it does                                               | What data it uses or sends                                                        |
|----------------|------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Home           | Landing page and marketing overview.                       | Static content and image asset.                                                   |
| Products       | Fetches medicine list and filters it by search text.       | GET <code>/api/products</code> , medicine objects, cart context.                  |
| ProductDetails | Shows the product id from the URL and provides navigation. | Route param <code>id</code> ; currently no live API fetch.                        |
| Cart           | Shows selected items, quantity controls, and checkout.     | CartContext data, total price, POST <code>/api/orders</code> .                    |
| Orders         | Loads and displays previous orders for the logged-in user. | GET <code>/api/orders</code> with Bearer token.                                   |
| Login          | Authenticates the user.                                    | POST <code>/api/auth/login</code> with email and password.                        |
| Register       | Creates a new user account.                                | POST <code>/api/auth/register</code> with name, email, password, confirmPassword. |
| About          | Explains the app and its purpose.                          | Static content and navigation links.                                              |

### 3.5 Styling system

The project keeps styling organized in a dedicated **css** folder instead of one giant stylesheet. That makes the app easier to maintain.

- **global.css** sets the theme, typography, layout helpers, buttons, cards, and inputs.
- **pages/\*.css** contains styles for each page.
- **components/\*.css** contains styles for reusable UI parts.

Important design detail: the app uses a calm teal-and-cream visual theme with shared

utility classes such as `card`, `btn`, `field`, `badge`, and `page`.

## 4. Backend Breakdown

### 4.1 `server.js`

The main backend file does four jobs:

- Loads environment variables from `backend/.env`.
- Creates the Express app and enables CORS and JSON parsing.
- Mounts the route groups for `auth`, `products`, and `orders`.
- Starts the server after the MongoDB connection is ready.

### 4.2 `config/db.js`

This file connects Mongoose to the database using `MONGO_URI` or `MONGODB_URI`. If the URI is missing, the app stops with a clear error message.

### 4.3 routes and controllers

#### Auth

- **`authRoutes.js`** exposes `/register` and `/login`.
- **`authControllers.js`** validates input, hashes passwords, checks login, and signs JWT tokens.

#### Products

- **`productRoutes.js`** exposes `GET /` and `POST /`.
- **`productController.js`** reads and writes product documents in MongoDB.

#### Orders

- **`OrderRoutes.js`** exposes `POST /` and `GET /`.
- **`OrderControllers.js`** creates new orders and fetches orders for the logged-in user.

#### Middleware

- **`authMiddleware.js`** checks the Bearer token, verifies it with JWT, and stores the decoded user on `req.user`.

## 5. End-to-End Data Flow

### 5.1 Registration flow

1. The user fills the register form with name, email, password, and confirmPassword.
2. The frontend sends that object to `POST /api/auth/register`.
3. The backend checks required fields, compares passwords, checks whether the email already exists, then hashes the password.
4. A new user is saved in MongoDB.
5. The response returns a success message and a safe user object.

## 5.2 Login flow

1. The user submits email and password.
2. The frontend sends the form data to `POST /api/auth/login`.
3. The backend finds the user by email and compares the password with bcrypt.
4. If valid, the backend creates a JWT token with the user id.
5. The frontend stores the user in AuthContext and stores the token in localStorage.

## 5.3 Browse products flow

1. The Products page calls `GET /api/products`.
2. The backend returns all saved product documents from MongoDB.
3. The page filters results locally using the search box.
4. Each product card receives name, price, company, and expiry.
5. When the user clicks Add To Cart, the whole product object is passed into CartContext.

## 5.4 Cart flow

1. CartContext stores the selected items in memory for the current session.
2. Each cart item has an id, name, price, company, expiry, and quantity.
3. The Cart page calculates the total price by multiplying item price by quantity.
4. When the user clicks Place Order, the page sends items and totalPrice to `POST /api/orders`.
5. The request includes `Authorization: Bearer <token>`.

## 5.5 Order flow

1. The backend auth middleware verifies the token and adds the decoded user id to the request.
2. The order controller saves the order with user, items, totalPrice, and status: "Pending".
3. Orders are saved in MongoDB with timestamps.
4. The Orders page calls `GET /api/orders` with the same token.

5. The backend returns only orders that belong to the logged-in user.

## 6. API Reference

| Method | Route              | Protected | Request Body                               | Response                                  |
|--------|--------------------|-----------|--------------------------------------------|-------------------------------------------|
| POST   | /api/auth/register | No        | { name, email, password, confirmPassword } | Message and created user object.          |
| POST   | /api/auth/login    | No        | { email, password }                        | JWT token and user object.                |
| GET    | /api/products      | No        | None                                       | Array of product documents.               |
| POST   | /api/products      | No        | Product data object                        | Created product document.                 |
| POST   | /api/orders        | Yes       | { items, totalPrice }                      | Created order document with success flag. |
| GET    | /api/orders        | Yes       | None                                       | Orders for the logged-in user.            |

Auth-protected routes expect an Authorization header in the form: Bearer <jwt token>

## 7. Database Models

### User

- name - required string
- email - required, unique string
- password - required hashed string

### Product

- name - required string
- price - required number
- company - required string

- Timestamps are enabled

- expiry - required string
- description - optional string
- stock - number with default 0
- Timestamps are enabled

## Order

- user - ObjectId reference to User
- items - array of item objects
- totalPrice - required number
- status - defaults to Pending
- Timestamps are enabled

## Cart item shape in frontend

- id and \_id for stable lookup
- name, price, company, expiry
- quantity for checkout math

## 8. Environment Variables

The backend reads configuration from `backend/.env`. The important values are:

- PORT - backend listening port
- MONGO\_URI or MONGODB\_URI - MongoDB connection string
- JWT\_SECRET - secret key used to sign and verify tokens

The frontend API client points to `http://localhost:5000/api` through `src/services/api.js`.

## 9. How to Run the Project

The app is split into a backend and frontend, so each part can be started separately.

Backend

```
cd my-react-app  
npm start
```

Frontend

```
cd my-react-app/frontend  
npm run dev
```

The backend listens on port **5000**. The frontend dev server runs on **3000**.

## 10. Notes and Future Improvements

- **ProductDetails** currently shows the route id and navigation, but it does not yet fetch a single product from the backend.
- The controller has a `getProductById` helper, but it is not currently wired to a route.
- **frontend/src/data/data.jsx** is currently empty, so the app depends on the MongoDB products collection instead of local mock data.
- Auth protection is split between frontend and backend. The frontend checks for a stored user, while the backend checks the JWT token.
- The cart is currently client-side state. That is fine for a simple app, but a future upgrade could save carts per user in MongoDB.

Summary: the app is a clean React + Express + MongoDB project with a clear shopping flow: register -> login -> browse products -> add to cart -> place order -> review orders.